

Presentation Q&A

Communications Systems Visualizer (ENGR 78)

Howard Wang

Kaw Moo

Charlie Schuetz

Spring 2026

Abstract

Likely audience questions with short **model answers** aligned to this repository (`main.py`, `web_app.py`, `pluto_live_app.py`, `src/`). Adjust wording for your demo path (hardware vs. simulation).

Goals and scope

Q1. Why a browser app instead of MATLAB, GNU Radio, or a Jupyter notebook?

Answer. A single `pip install -r requirements.txt` and `python main.py` gives a tabbed UI in Chrome or Safari with no license server, no cell re-run order, and no radio toolchain for the *core* course path. Dash + Plotly match how we wanted linked sliders and multiple panes updated together for teaching.

Q2. What parts of ENGR 78 does the tool cover, and what did you deliberately leave out?

Answer. Covered: PCM-style sampling/quantization intuition, line codes + PSD, raised-cosine ISI and eye segments, M -ary PAM, passband ASK/BFSK/QPSK/16-QAM views, Shannon capacity vs. SNR, binary entropy / BSC rate, and a tiny repetition-code nearest-codeword toy. Left out: full FEC chains, OFDM, multipath channel models in the *main* app, calibrated RF lab measurements, and production-grade receiver acquisition.

Q3. Who is the primary user, and how long does it take to get running?

Answer. Primary users are students and instructors doing quick explorations on a laptop. Install dependencies once, then start the server; the browser opens to `127.0.0.1:8050`. Optional Pluto live is a second command on port 8051.

Q4. How is this different from static plots in the lecture slides?

Answer. Sliders and bit strings recompute traces immediately, so users see how bandwidth, noise, roll-off, and modulation order trade off in *linked* time, frequency, and state-space views instead of fixed figures.

Architecture and implementation

Q5. Walk through the data flow: user moves a slider $\rightarrow$ what runs $\rightarrow$ what goes to Plotly?

Answer. Dash matches control `Inputs` to an `@app.callback` in `web_app.py`. That Python function builds NumPy arrays and returns Plotly `figure` objects (and small `html` stat blocks). Dash serializes the figure JSON to the browser; Plotly.js renders it.

Q6. Why Dash and Plotly instead of another web stack or a desktop GUI?

Answer. Dash gives declarative layouts and callbacks with minimal front-end code for a course-scale project. Plotly gives interactive zoom/pan and consistent styling across tabs. A desktop toolkit (Qt, etc.) would work but adds packaging friction for peers on different OSes.

Q7. Where does the heavy numerics live, and could that split be cleaner?

Answer. Almost all signal generation for the main dashboard lives in `web_app.py` (helper functions + callbacks). A cleaner split would move pure DSP into `src/` modules and keep callbacks thin; we prioritized one place to read for the class demo.

Q8. How do you keep the UI responsive when FFTs or waveforms are expensive?

Answer. We keep vector lengths modest (fixed oversampling, bounded bit strings, limited eye traces). Callbacks return only the traces needed for the active tab. For Pluto live, a background thread does DSP and the UI polls at an interval so the browser thread is not buried in FFTs.

Q9. How would you add unit tests for the signal math?

Answer. Extract functions like `line_code`, `raised_cosine`, `carrier_waveform`, and `spectrum_db` into importable modules, then assert known outputs on golden inputs (e.g., all-ones NRZ spectrum shape, energy in main lobe). For Pluto sync, regression-test `synchronize` on synthetic IQ with noise.

Main dashboard (PCM, line coding, ISI, carrier, capacity)

Q10. For PCM, what exactly is being plotted versus true Nyquist sampling of a bandlimited message?

Answer. The message is a *synthetic* two-tone sinusoid whose highest frequency is tied to the message bandwidth slider B_m . Samples use $f_s = (\text{multiplier}) \times B_m$; we plot message, samples, and uniform quantizer output and report $R_b = n f_s$. It illustrates sampling + quantization + bit rate, not a rigorous sampling theorem proof with an arbitrary bandlimited waveform.

Q11. How do you compute PSD for line codes?

Answer. For the line-code waveform, we remove the mean, apply a Hann window, zero-pad to $n_{\text{fft}} = 2^{\lceil \log_2 \max(512, 4N) \rceil}$ (code: `spectrum_db`), take one-sided `rfft`, square magnitude, normalize to peak 0 dB, then plot $10 \log_{10}(\cdot)$ vs. frequency in cycles per bit time.

Q12. In the ISI/eye tab, what pulse and receiver assumptions are you using?

Answer. Symbols are random M -ary PAM on a raised-cosine pulse (`raised_cosine` with roll-off β). Optional `add_controlled_isi` mixes scaled early/late copies to mimic ISI. AWGN is added to the convolved waveform. Eye segments overlay two symbol periods of the *same* noisy waveform; the matched filter at the receiver is not a separate second filter in code—the visualization is diagnostic.

Q13. For QPSK or 16-QAM, is the spectrum analytic or real passband?

Answer. The stored carrier waveform `y` is **real** passband: I/Q symbols modulate `cos` and `sin` at a fixed integer cycles-per-symbol rate inside `carrier_waveform`. The spectrum plot is an FFT magnitude of that real signal (scaled), so you see symmetric sidebands about the effective carrier, not a one-sided complex baseband spectrum.

Q14. How is “throughput” defined in the carrier tab?

Answer. It is a **nominal information rate** from the controls: $\text{throughput} = R_s \times (\text{bits per carrier symbol})$ with a second flat line for the M -ary PAM bit rate $R_s \log_2 M$. Cumulative bits on the secondary axis is `carrier_rate` $\times t$. It is *not* a measured decoder output on the main dashboard.

Q15. How does the Shannon capacity curve relate to the SNR sliders elsewhere?

Answer. The capacity tab has its **own** bandwidth and SNR sliders for $C = B \log_2(1 + \text{SNR})$. They are didactic: moving carrier or ISI noise does not automatically move that operating point. You can verbally tie them: “same Shannon limit idea, separate controls for clarity.”

Q16. What simplifying assumptions does the repetition-code demo make?

Answer. Hard bit errors on a 3-bit word with a simple UI; decoding picks codeword 000 or 111 by **minimum Hamming distance**. No soft LLRs, no interleaving, no realistic channel model—just a visualization of redundancy and majority-like intuition.

Optional Pluto live path

Q17. Why a second process on port 8051?

Answer. Two Dash apps each call `app.run` with one port. Running both in one process would require mounting sub-apps or dispatching routes; separate files keep the default class visualizer (`main.py`, 8050) independent from the optional radio demo (8051).

Q18. What does “Simulation (no radio)” do?

Answer. `PlutoInterface` returns tiled transmit IQ plus small complex AWGN instead of calling `sdr.rx()`. The same `RealtimeEngine` DSP chain runs so the UI matches hardware mode without libiio hardware.

Q19. What are the default RF settings for real Pluto, and who chose them?

Answer. In `pluto_interface.py`: LO = 915 MHz, sample rate and RF BW = 1 MHz, TX gain -30 dB, RX AGC `slow_attack`, RX buffer 2^{14} samples, cyclic TX buffer. These are safe classroom defaults for loopback/cable demos, not a regulatory approval.

Q20. How does frame synchronization work, and if sync fails?

Answer. `synchronization.py`: Barker preamble in the frame, cross-correlation for start, coarse frequency from repeated preamble, Gardner timing (with a fallback path without Gardner), then phase on the preamble. If no usable payload symbols are produced, the engine skips updating the latest result and the dashboard stays on placeholders until a good buffer arrives.

Q21. Does the dashboard show every RX sample?

Answer. No. The engine keeps only the **latest** buffer for the UI; the spectrum trace is decimated for plotting. Many DSP buffers between browser ticks are dropped from view.

Q22. Is any IQ sent to the cloud?

Answer. No. Everything stays in-process; the browser receives figure JSON from your local Dash server, not raw IQ uploads.

Q23. What would you change for a real over-the-air demo?

Answer. Pick an authorized ISM band and antenna, set legal ERP with attenuation and duty cycle, verify image rejection and LO leakage, use proper filtering and maybe lower symbol rate, and document cable vs. antenna loopback so grading expectations stay clear.

Limitations, honesty, and extensions

Q24. What are the biggest modeling limitations?

Answer. Main app: idealized waveforms, AWGN only where noise appears, no multipath in the primary tabs, repetition toy instead of real FEC. Pluto path: single-buffer diagnostics, default RF not tuned per venue, BER/EVM are simulation-level unless you calibrate against a reference.

Q25. If you had two more weeks, what would you ship first?

Answer. Reasonable picks: (1) export traces to CSV for lab reports; (2) split `web_app.py` DSP into tested modules; (3) Pluto UI knobs for LO/rate/gain; or (4) saved “lecture presets” that restore a full tab state.

Q26. How would you validate Pluto EVM/BER?

Answer. Cable loopback at low power, known fixed payload bits, compare demodulated bits to TX reference in simulation first; on hardware, step SNR and compare to analytic Q-function curve for QPSK in AWGN for order-of-magnitude agreement.

Q27. Could another course reuse this repo?

Answer. Yes, with a short “instructor README”: Python version, `requirements.txt`, ports 8050/8051, optional libio/Pluto install, which tabs map to which lecture week, and a disclaimer that RF defaults are for education not deployment.

Build: `make -C docs presentation_qa.pdf` or `make -C docs qa`.